SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 3-QUESTIONS

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)

Part A – Avalanche and Fixed-Length Digests

Shared Group Work

Together:

- Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)
- Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)
- Decide how "avalanche" will be measured
- Run and test the experiment

Student A – Design & Mathematical Analysis

Student A must individually:

- Explain what each property means as an attack scenari
- Derive the expected number of output bits that change when one input bit is flipped in an ideal hash
- Explain mathematically why the toy hash fails the avalanche and second pre-image properties
- Explain why a fixed-length digest must have collisions

Student B – Experimental Validation

Student B must individually:

- Design the experimental inputs (single-bit flips, similar messages, long and short messages)
- Instrument the code to count the number of changed output bits
- Test at least three message sizes and a minimum number of trials per size
- Compare predicted and observed bit-change averages
- Explain any discrepancy

Part B – Collision Effort and the Birthday Bound

Shared Group Work

Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32).

Student A – Experimental Investigation

Student A must:

- Design the collision-search runs and repeat each several times
- Record the number of hashes computed before the first collision
- Investigate increasing values of k
- Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning

Student B must:

- Derive why the expected effort is roughly 2^(k/2), not 2^k
- Explain the difference between finding any collision and finding a collision with a specific message
- Explain how this determines the minimum safe digest length
- Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

- Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)
- Implement the naive MAC H(key || message) and the HMAC construction
- Design tamper scenarios (modified message, modified message and recomputed digest, appended data)
- Conduct the experiments

Student A – Attack and Construction Investigation

Student A is individually responsible for:

- Showing that an attacker who can change both the file and the published digest defeats a plain hash
- Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash
- Recording what the attacker needs to know and what the attacker does not need to know
- Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis

Student B is individually responsible for:

•Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

- Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither
- Explaining why a MAC cannot provide non-repudiation
- Verifying through tests that HMAC detects every tamper scenario you designed

Joint Design Question

Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?

Both students must be able to defend the answer.

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)

Shared Group Work

Together:

- Implement RSA signature generation and verification using hash-then-sign (small keys for measurement)
- Implement a DSA-style (or ECDSA-style) signature using toy parameters
- Test valid signatures, modified messages, modified signatures and wrong public keys
- Conduct the nonce experiment described below

Student A – Algorithmic Derivation

Student A must individually:

- Explain why the message is hashed before signing and what would go wrong with signing the raw message
- Prove why verification succeeds for a correctly generated RSA signature ($s = H(m)^d \bmod n$ and $s^e \equiv H(m) \bmod n$)
- Derive the DSA/ECDSA verification equation and show why it recovers the signer's value
- Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence

Student B must individually:

- Design the test messages and tamper cases and record accept/reject results
- Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures
- Demonstrate a textbook (no hash) RSA signature forgery (for example, multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2)
- Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion

Together answer:

A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation, and what could the signer claim to deny it?

Consider key compromise, timestamping, and the role of a trusted third party.

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)

Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.

Shared Group Work

Together:

- Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)
- Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL
- Implement a chain-validation procedure that follows the steps a client must perform
- Verify the procedure on valid and invalid chains

Student B – Trust Model & Validation Logic

Student B must individually:

- Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)
- Explain what each step protects against
- Explain why trust in the root cannot itself be verified cryptographically
- Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful

Student A – Failure-Case Experiments

Student A must individually:

- Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate
- Record which step of validation rejects each case
- Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually performed
- Discuss what a "fail-open" revocation policy means for security

Joint Engineering Decision

Together determine:

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Shared Group Work

Together:

- Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)
- Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets
- Include timestamps, ticket lifetimes, session keys and authenticators
- Test the normal flow and the attack/failure cases below

Student A – Protocol Design

Student A must individually:

- Draw the message-sequence diagram and label what is encrypted with which key
- Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session
- Explain the purpose of the authenticator and the timestamp
- Explain how mutual authentication is achieved
- Identify which party can read which part of each message

Student B – Attack and Failure Evidence

Student B must individually:

- Demonstrate a replay of a captured authenticator with and without replay protection
- Demonstrate an expired ticket and a clock-skew failure
- Demonstrate what happens if the KDC is unavailable
- Record accept/reject outcomes and the reason for each
- Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow

Cross-Understanding Requirement

Student B must be able to explain:

Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:

What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay?This deliberately tests understanding across the assigned responsibilities.

Q6 – Advanced Security Mini-Investigations: RSA, DSA/DSS, ECDSA, ElGamal and Schnorr (7 Marks)

This question provides particularly clear individual evidence.

Student A – Hash Weakness and Signature Forgery

Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- Explain the difference between a collision attack and a second pre-image attack
- Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)
- Explain why this attack requires the attacker to prepare both messages in advance
- Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509
- Explain the role of digest length and algorithm choice in deciding what a CA may use
- Recommend a hash policy for signatures and justify it

Student B Verification

Student B must provide a short explanation of:

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.

The student must:

- Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment
- Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)
- Analyse the single point of failure of Kerberos and the trust-anchor risk in X.509
- Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI
- Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr.

• **Comparison requirement**: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions.

• **Schnorr protocol**: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.

• **ElGamal signature scheme**: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Student B independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- Where the designs agree?
- Where they differ?
- Strengths and weaknesses of each

- Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key)
- Which ideas should be retained?

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

Stage 3 – Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- Threat model (who the attacker is and what the attacker can do)
- Security goals and the mechanism that achieves each
- Key and certificate management (issuance, storage, rotation, revocation)
- Message flow / protocol (diagram) for signing, distributing and verifying an update
- Behaviour when the pump is offline or its clock is wrong
- Complete pseudocode for signing and for verification

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation

Student A must:

- State what each attack would need to succeed (e.g., 2^(n/2) work for a collision, forging a signature, compromising a key)
- Show, goal by goal, why the final design prevents or limits each attack
- Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3
- Identify the residual risks that remain after the design

Student B – Experimental Validation

Student B must:

- Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)
- Implement and instrument the final verification procedure
- Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop
- Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata

Stage 5 – Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation

Together determine:

- Which of integrity, authentication and non-repudiation is guaranteed by which part of the design
- What costs the design introduces (verification time, certificate storage, key-management burden)
- Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)
- Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT – 3

EVALUATION, DESIGN AND DEFENCE OF CRYPTOGRAPHIC HASHES, DIGITAL SIGNATURES AND SECURE AUTHENTICATION FRAMEWORKS

Course: Cryptography and Network Security
Course Code: CSA5102 | Course Outcome: CO3
Total Marks: 50

STUDENT DETAILS	
Name	R. KALAI AMUDHAN
Register Number	192511292
Programme	B.E. – Computer Science and Engineering
Course name	CRYPTOGRAPHY AND NETWORK SECURITY
Course code	CSA5102
Faculty Name	Dr. E Anbalagan

Q1 – WHAT MAKES A HASH FUNCTION TRUSTWORTHY?

Part A – Student A: Design and Mathematical Analysis

Property	Attack scenario	Generic cost for an n-bit digest
Pre-image resistance	Given a stored digest h (for example a password hash), the attacker finds any m with H(m) = h.	about 2^n hashes
Second pre-image resistance	Given a signed document m, the attacker finds a different m' with H(m') = H(m), so the same signature validates.	about 2^n hashes
Collision resistance	The attacker prepares two messages m and m' in advance (a benign and a malicious contract) with H(m) = H(m') and gets the benign one signed.	about 2^(n/2) hashes (birthday bound)
Avalanche effect	A change of one input bit must not leave a pattern: otherwise similar messages give related digests and the attacker can steer the output.	each output bit flips with probability 1/2

Expected bit change for an ideal hash. Let the digest have n bits. For an ideal hash, flipping one input bit makes the new digest an independent uniformly random n-bit string. The indicator X_i that output bit i changes has P(X_i = 1) = 1/2, so E[X] = sum E[X_i] = n/2 and, with independence, Var[X] = n/4 and the standard deviation is sqrt(n)/2. For n = 32 the expectation is 16 bits (SD 2.83); for n = 256 it is 128 bits (SD 8).

Why the toy hash fails. Byte-sum: H(m) = (sum of bytes) mod 2^32. Flipping bit j of a byte changes the sum by +2^j or -2^j, so the digest changes by a small carry chain and only about 2 bits change on average (observed 2.07), never 16. Also H depends only on the multiset of bytes: swapping two bytes, or adding 1 to one byte and subtracting 1 from another, leaves H unchanged. A second pre-image is therefore found in O(1) work instead of 2^32. XOR-fold flips exactly one output bit for one input bit flip (observed 1.00, SD 0) and is invariant under permutation of 4-byte words and under XOR-ing the same value into two words.

Second pre-image test	Message	Byte-sum digest	XOR-fold digest	SHA-256 (first 32 bits)
Original	PAY 100 TO DR A	885	1730021444	3312681553
Swap first two bytes	APY 100 TO DR A	885	1980696644	2992558998
Add 1 to byte 3, subtract 1 from byte 4	PAZ?100 TO DR A	885	-	-

The byte-sum digest is identical (885) for three different messages, while SHA-256 changes completely, so the toy hash fails second pre-image resistance.

Fixed length means collisions. A hash maps an unbounded set of messages to 2^n digests. Among the 2^(n+1) messages of n+1 bits, the pigeonhole principle forces at least one digest to have two or more pre-images; for all messages of m > n bits, some digest has at least 2^(m-n) pre-images. Collisions therefore exist; security means only that they are infeasible to find.

Part A – Student B: Experimental Validation

Inputs: random messages of 16, 64, 256 and 1024 bytes; for each size 1,000 trials, each flipping one random input bit and counting the changed digest bits (popcount of the XOR of the two digests). Compared functions: byte-sum-32, XOR-fold-32, SHA-256 truncated to 32 bits and full SHA-256.

Hash	Size (bytes)	Trials	Observed mean changed bits	Predicted n/2	Observed SD	Predicted SD sqrt(n)/2
Byte-sum (32-bit)	16	1000	2.07	16.0	1.52	2.83
XOR-fold (32-bit)	16	1000	1	16.0	0.0	2.83
SHA-256 truncated to 32 bits	16	1000	16.03	16.0	2.84	2.83
SHA-256 (256-bit)	16	1000	128.01	128.0	7.97	8.0
Byte-sum (32-bit)	64	1000	2.05	16.0	1.61	2.83
XOR-fold (32-bit)	64	1000	1	16.0	0.0	2.83
SHA-256 truncated to 32 bits	64	1000	16.01	16.0	2.86	2.83
SHA-256 (256-bit)	64	1000	127.89	128.0	8.13	8.0
Byte-sum (32-bit)	256	1000	2.07	16.0	1.59	2.83
XOR-fold (32-bit)	256	1000	1	16.0	0.0	2.83
SHA-256 truncated to 32 bits	256	1000	15.91	16.0	2.9	2.83
SHA-256 (256-bit)	256	1000	127.9	128.0	8.22	8.0
Byte-sum (32-bit)	1024	1000	2.04	16.0	1.52	2.83
XOR-fold (32-bit)	1024	1000	1	16.0	0.0	2.83
SHA-256 truncated to 32 bits	1024	1000	15.88	16.0	2.92	2.83
SHA-256 (256-bit)	1024	1000	128.07	128.0	7.87	8.0

Discrepancy analysis: SHA-256 and truncated SHA-256 match the prediction within sampling error (for 1,000 trials the standard error of the mean is about SD/31.6, i.e. 0.09 bits for 32-bit output and 0.25 bits for 256-bit output). The toy hashes miss the ideal mean by 14 bits (byte-sum) and 15 bits (XOR-fold) because of the structure explained above, and the result does not change with message length, which shows that the weakness is structural and not statistical.

Part B – Student A: Collision Experiments

The collision finder takes the first k bits of SHA-256, hashes messages (random 8-byte prefix plus counter) and stores digests in a dictionary until a repeat occurs; it counts hashes computed. 20 repetitions for k = 16, 20, 24, 28 and 10 repetitions for k = 32.

k (bits)	Runs	Min	Mean	Max	Predicted 1.2533 x 2^(k/2)	2^(k/2)	Mean / 2^(k/2)
16	20	81	299	732	321	256	1.17
20	20	110	1189	2556	1283	1024	1.16
24	20	1013	5271	11834	5134	4096	1.29
28	20	6824	18724	42672	20534	16384	1.14
32	10	46887	88937	164498	82136	65536	1.36

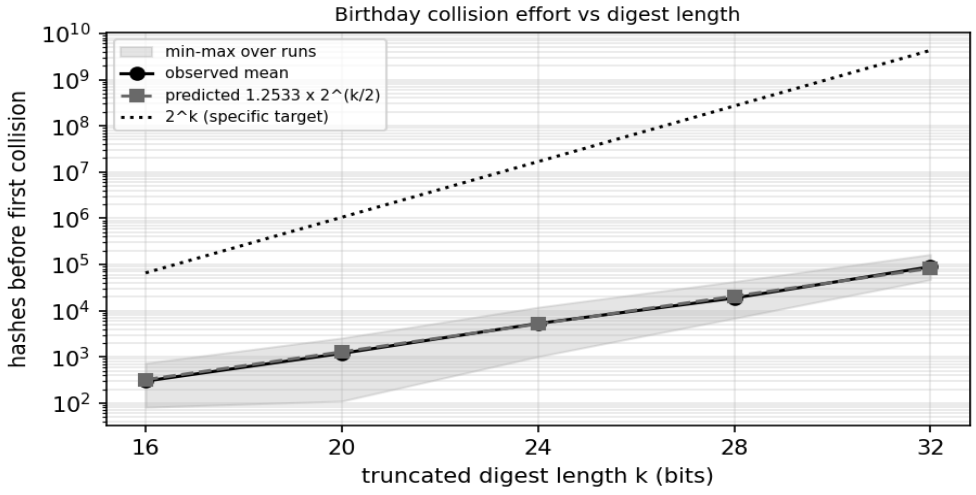


Figure 1 – Hashes needed for the first collision versus k (log scale). The observed mean follows 2^(k/2); a specific-target search follows 2^k.

k (bits)	Mean hashes to hit one specific target (20 runs)	2^k
8	284	256
12	4637	4096
16	67731	65536

Part B – Student B: Birthday-Bound Reasoning

Derivation: with N = 2^k equally likely digests, after t hashes there are t(t-1)/2 pairs, each colliding with probability 1/N. The probability of no collision is about exp(-t^2/(2N)), so a collision has probability 1/2 at t about 1.18 sqrt(N) and the expected number of hashes is sqrt(pi N / 2) = 1.2533 x 2^(k/2). The measured ratios of 1.14 to 1.36 agree with the constant 1.2533.

Any collision versus a specific target: any collision involves all pairs and needs about 2^(k/2) hashes, whereas matching one fixed digest needs about 2^k hashes because each trial succeeds with probability 2^-k (measured 67,731 against 65,536 for k = 16). Minimum safe length: for a security level of s bits against collisions the digest must have at least 2s bits, so 128-bit collision resistance needs 256-bit digests (SHA-256, SHA3-256), and a digest shorter than 224 bits is not acceptable for new signatures.

MD5 (128-bit) offers only 2^64 generic collision work and was broken far below that by differential cryptanalysis (practical collisions in 2004 and chosen-prefix collisions used in 2008 and 2012 against certificate authorities). SHA-1 (160-bit) offers 2^80 generic work; the SHAttered collision in 2017 needed about 2^63 operations, so both were retired from signature and certificate use.

Individual verification: Student A can explain the birthday bound with the table and graph above; Student B can explain the avalanche counting experiment (flip one bit, XOR digests, popcount, average over 1,000 trials).

Q2 – INTEGRITY IS NOT AUTHENTICATION: HASH, MAC AND HMAC

Shared implementation

A toy Merkle–Damgård hash was implemented: 64-bit chaining value, 16-byte blocks, compression function f(s, block) built from XOR, multiplication by a 64-bit constant, xor-shift and rotation applied to the two 8-byte halves, IV = 0x0123456789ABCDEF, and padding of 0x80, zero bytes and an 8-byte bit length. The naive MAC is H(key || message) and HMAC follows RFC 2104 (block size 16): HMAC(K, m) = H((K xor opad) || H((K xor ipad) || m)). The HMAC code is generic in the hash; with SHA-256 and block size 64 it matches Python hmac exactly (check = Yes).

Student A – Attacks

1. Plain hash is not authentication. Original download: digest 805f28d1ccd729b7. An attacker who controls the web page replaces the file (amount=900&to=mallory) and publishes its digest 19f846e286b0a52d; the verifier recomputes and compares, and the check passes (verification result: True). A hash gives integrity only against accidental change, because the reference value is not protected by a secret or a signature.

2. Length-extension attack on H(key || message). Message m = "amount=100&to=alice", secret key of 10 bytes, tag T = 80bcdebf472efb6b. The attacker knows m, T and the key length (a guess in the range of possible lengths), but not the key. Since the digest is the final chaining value, the attacker resumes the compression from T as the state and processes the extra data "&to=mallory" with offset = length of key || m || glue padding. The forged message is m || glue || "&to=mallory" with forged tag 45e0188241d9b86c. The verifier, which holds the key, computes H(key || forged message) = 45e0188241d9b86c; match = Yes. The forged message in hex is: 616d6f756e743d31303026746f3d616c6963658000000000000000000000000000000e826746f3d6d616c6c6f7279.

Key length guessed by attacker	Forged tag accepted?
4	No
8	No
10	Yes
12	No
16	No

What the attacker needs: the original message, the tag, the length of the secret key (tried by guessing, as the table shows only the true length 10 works), and knowledge of the hash algorithm and padding rule. What the attacker does not need: the key. Cause: the Merkle–Damgård construction outputs its full internal state, and the padding is a deterministic function of the length, so the tag is exactly the state the verifier would reach after processing the original message and padding; the attacker can simply continue from it.

Student B – Comparative Analysis

HMAC structure: the key is padded with zeros to the block size (or hashed first if longer), xor-ed with ipad (0x36 repeated) for the inner hash and with opad (0x5c repeated) for the outer hash. The inner digest is never released; the outer hash processes a fresh, key-dependent block and the inner digest, so the final output is not a chaining value of a message the attacker can extend. Extending the tag would require the inner hash value, which is hidden by the outer hash under the secret opad-key block.

Mechanism	Integrity against attacker with no key	Authentication of sender	Non-repudiation	Result of the length-extension test
Plain hash H(m)	Only if the digest is delivered over a trusted channel	No	No	Not applicable
Naive MAC H(key‖m)	Yes for fresh messages	Yes (shared secret)	No	Forgery accepted
HMAC	Yes	Yes (shared secret)	No	Forgery rejected

A MAC cannot give non-repudiation because both parties hold the same key: the receiver could have created any tag, so a tag does not prove to a third party which of the two produced the message. Only an asymmetric signature, where the private key is held by one party only, can do that.

Tamper scenario	Plain hash detects?	Naive MAC detects?	HMAC detects?
Modify message only	Yes	Yes	Yes
Modify message and recompute digest/tag (no key)	No	Yes	Yes
Append data (length extension, no key)	Yes	No	Yes
Flip one bit of the tag	Yes	Yes	Yes

Test: does the verifier accept the forgery?	Accepted
Plain hash (attacker replaces file and digest)	Yes
Naive H(key‖m) against length extension	Yes
HMAC against the same extended message	No

Row 2 shows the plain hash fails when the attacker also recomputes the digest (no secret involved); row 3 shows only HMAC detects the appended data. HMAC detected all four designed scenarios.

Joint Design Decision

Download verification: use a digital signature (for example Ed25519 or ECDSA P-256 over SHA-256) by the distributor, with the public key or certificate obtained out of band or through a trusted store; a digest on the same web page is not enough because the attacker who can change the file can change the digest. Internal messages between parties that share a key: HMAC-SHA-256 (or an AEAD such as AES-GCM when confidentiality is also needed) with per-pair keys, a counter or nonce inside the message to stop replay, and constant-time tag comparison. Secret items: the HMAC keys and the signing private key; the file, digest and public key are public. Residual risks: compromise or leakage of a shared key (and no non-repudiation), compromise of the signing key, replay of old valid messages, and a user who skips verification.

Q3 – DIGITAL SIGNATURES: HASH-THEN-SIGN AND WHAT CAN BREAK THEM

Student A – Derivations

Why hash first: (1) the raw message must be smaller than the RSA modulus and signing a long message needs many expensive operations, while a fixed 256-bit digest is signed once; (2) integrity is bound to the whole message through the collision-resistant hash; (3) it destroys the algebraic structure of raw RSA, preventing the multiplicative forgery and blinding forgeries shown below; (4) the signature is not computed over attacker-chosen group elements.

RSA correctness: with n = pq, ed = 1 + k phi(n), signing s = H^d mod n and verifying s^e = H^(de) = H^(1 + k phi(n)) = H x (H^phi(n))^k = H (mod n) by Euler's theorem when gcd(H, n) = 1 (and also otherwise by the Chinese Remainder Theorem). Hence verification succeeds exactly when the recomputed hash equals the recovered value.

DSA verification: with signing s = k^-1 (H + x r) mod q and r = (g^k mod p) mod q, set w = s^-1 mod q, u1 = H w, u2 = r w. Then g^u1 y^u2 = g^((H + x r) w) = g^(k) (mod p) since (H + x r) w = k, and taking mod q gives r, so v = r. ECDSA is identical in an elliptic-curve group: R = u1 G + u2 Q = (H + r d) s^-1 G = k G, and the check is x(R) mod n = r.

Step	Security goal contributed
Hash the message	Integrity: any change alters the digest
Sign the digest with the private key	Authentication: only the key holder can produce the value
Verify with the public key bound to an identity (certificate)	Authentication of the identity; with exclusive key control, non-repudiation (conditions below)

Student B – Experiments

Case	RSA-512 hash-then-sign	DSA (p=128 bit, q=64 bit)
Valid message and signature	Accepted	Accepted
Modified message	Rejected	Rejected
Modified signature	Rejected	Rejected
Wrong public key	Rejected	Rejected

Nonce reuse in DSA: two messages ("Dose 10mg", "Dose 90mg") signed with the same k give the same r (check = Yes). From s1 = k^-1 (H1 + x r) and s2 = k^-1 (H2 + x r): k = (H1 - H2)(s1 - s2)^-1 mod q, then x = (s1 k - H1) r^-1 mod q. Recovered k equals the real nonce: Yes; recovered private key equals the real key: Yes; g^x = y: Yes. The same algebra applies to ECDSA, which is how real private keys have been stolen from devices with weak random generators.

Textbook (no hash) RSA forgery: with m1 = 12345 and m2 = 67890, s1 = m1^d and s2 = m2^d are valid signatures; s1 s2 mod n is a valid signature on m1 m2 = 838102050 (verification result: Yes), obtained without the private key. Blinding: for a target message 424242 the attacker asks the signer to sign m x r^e for a random r, divides the result by r and obtains a valid signature on the target (result: Yes). With hash-then-sign these attacks fail because the attacker would have to find messages whose hashes multiply to the target hash.

Conclusion: the mathematics of RSA and DSA is correct, but the implementation decisions (no hashing, reusing or predicting nonces) make the scheme insecure. Security depends on the whole construction, not only the algorithm.

Joint Conclusion – Non-Repudiation

A valid signature proves that the private key was used; it supports non-repudiation only if (i) the key was generated and held exclusively by the signer (preferably in a token or HSM with user authorisation), (ii) the public key is bound to the signer by a trustworthy certificate valid at signing time, (iii) the time of signing is proven by a trusted timestamp so that later key compromise or revocation does not invalidate old signatures, and (iv) the hash and signature algorithms were still sound at that time. A signer could try to deny by claiming key theft or malware (key compromise), saying that the key was revoked or expired before signing, claiming the signed content was different (display problem, collision), or claiming nonce/implementation flaws. A trusted third party (CA, timestamping authority, and possibly an audit or notary service) supplies the evidence that limits these claims; the signer remains liable only if the key management policy is followed.

Q4 – TRUST THROUGH CERTIFICATES: X.509 CHAIN VALIDATION

A three-level hierarchy (Root CA, Intermediate CA, end-entity clinic.example) was built with the Python cryptography library (ECDSA P-256 keys, SHA-256 signatures). The chain-validation procedure was implemented by hand (Appendix B, function validate).

Subject	Issuer	Serial (low 24 bits)	Validity	CA flag
CN=Root CA	CN=Root CA	a29820	2021-11-07 to 2036-11-03	Yes
CN=Intermediate CA	CN=Root CA	2a3a1d	2024-11-06 to 2031-11-05	Yes
CN=clinic.example	CN=Intermediate CA	b96d75	2026-10-07 to 2027-11-06	No

Student B – Validation algorithm

```
validate(chain[leaf..], trusted_roots, host, now, revocation):
 S1  anchor: issuer of the last certificate must match a trusted root          (no unknown CA)
 S2  for each link: cert.issuer == parent.subject                              (right parent)
 S3  for each link: verify cert.signature with the parent public key           (not forged / altered)
     verify the root self-signature                                            (consistent anchor)
 S4  for each certificate: not_before <= now <= not_after                      (no stale or early cert)
 S5  every issuer certificate: basicConstraints CA = TRUE, pathLen respected   (leaf cannot act as CA)
 S6  every issuer certificate: keyUsage contains keyCertSign                   (key allowed to sign
certs)
 S7  host must be in leaf subjectAltName (DNS), wildcard rules applied         (right identity)
 S8  no certificate in the chain revoked (CRL lookup or OCSP, soft/hard fail)  (compromised key)
 return ACCEPT only if every step passes
```

Why trust in the root cannot be verified cryptographically: a root certificate is self-signed, so its signature proves only that the owner has the matching private key, not that the owner is trustworthy. Trust comes from out-of-band distribution (operating system or browser trust stores, audited CA programs). X.509 binds identity to key because the CA signs the subject name together with the public key; a signature is meaningful only because the verifier can say whose key verified it.

Student A – Failure cases

Failure case	Accepted?	Step that decided
Valid chain	Yes	accepted
Expired end-entity certificate	No	S4 validity period (cert 0: CN=clinic.example)
Not-yet-valid end-entity certificate	No	S4 validity period (cert 0: CN=clinic.example)
Wrong issuer (signed by rogue CA, name Rogue CA)	No	S2 issuer/subject name match (cert 0)
Right issuer name but signed with rogue key	No	S3 signature verification (cert 0)
Tampered field (certificate bytes modified after signing)	No	S3 signature verification (cert 0)
Hostname mismatch (valid chain, wrong name requested)	No	S7 hostname match (SAN=['clinic.example'])
End-entity certificate used as a CA	No	S5 basicConstraints CA flag (cert 1 is not a CA)
Untrusted root (anchor not installed)	No	S1 trust anchor: issuer of top certificate is not a trusted root
Revoked end-entity, revocation checked	No	S8 revocation (serial 6d75 listed)
Revoked end-entity, revocation NOT checked (fail-open)	Yes	accepted

Property	CRL	OCSP
Freshness	As old as the last published list (hours to days)	Near real time (per query or stapled response lifetime, minutes to days)
Size	Grows with the number of revoked certificates	Small, one response per certificate
Privacy	Good: client downloads the same list for everyone	The responder learns which sites the client visits (stapling avoids this)
Availability	Cacheable and works offline once downloaded	Responder must be reachable, or the client must decide fail-open or fail-closed

The last two rows of the failure table show the point: the revoked certificate is rejected only when the revocation step is actually executed; with the check disabled (or when the CRL/OCSP server is unreachable and the client fails open) the same revoked certificate is accepted. A fail-open policy therefore lets an attacker who holds a stolen key and can block the revocation traffic keep using it; fail-closed or OCSP Must-Staple removes this gap but needs reliable infrastructure.

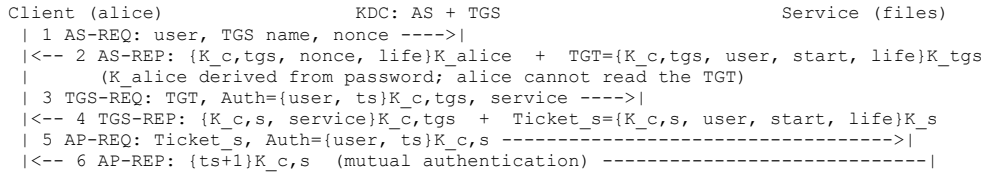
Joint Engineering Decision

Damage is limited by the certificate validity period, path-length and name constraints at the intermediate, CA key protection in HSMs, offline roots, and Certificate Transparency logging so that mis-issuance is discovered. The damage persists until revocation information reaches clients (CRL interval or OCSP response lifetime, hours to days) or, when revocation is not checked, until the certificate expires. To shorten the window: short-lived leaf certificates (days), OCSP stapling with Must-Staple, pushed revocation lists (for example browser CRLsets), automated renewal, monitoring of CT logs, and the ability to remove a compromised root or intermediate from the trust store quickly.

Q5 – KERBEROS: AUTHENTICATION WITHOUT SENDING PASSWORDS

The simulation has a client (alice), a KDC with an Authentication Server and Ticket-Granting Server, and a file service. Tickets and authenticators are sealed with AES-GCM (authenticated encryption), the user key is PBKDF2 of the password (2,000 iterations in the toy), timestamps come from a simulated clock, the ticket lifetime is 3,600 s for service tickets and 36,000 s for the TGT, and the allowed clock skew is 300 s. Code in Appendix B.

Student A – Protocol design



Item	Encrypted with	Who can read it
AS-REP session part {K_c,tgs, nonce}	K_alice (password-derived)	Only alice (and KDC)
TGT	K_tgs (KDC secret)	Only the KDC; alice just carries it
Authenticator to TGS {user, ts}	K_c,tgs	Alice and the TGS
Service session part {K_c,s}	K_c,tgs	Alice and the TGS
Service ticket	K_s (service secret)	Only the service (and KDC)
Authenticator to service, AP-REP	K_c,s	Alice and the service

The TGT lets the user obtain many service tickets without re-entering the password: the long-term key K_alice is used only once per session to decrypt the AS reply, which limits exposure of the password-derived key. The authenticator proves, with a fresh timestamp encrypted under the session key, that the sender knows the ticket session key at this moment; the timestamp makes a captured authenticator useful only inside a short window and, with a replay cache, only once. Mutual authentication: the service returns ts+1 encrypted with K_c,s, which only a holder of the ticket key (that is, the genuine service) can produce.

Student B – Attack and failure evidence

Normal flow step	Outcome	Check
AS exchange	alice sends name + nonce; receives TGT and {K_c,tgs, nonce}K_alice	nonce echoed = True
TGS exchange	TGT + authenticator{K_c,tgs}	service ticket issued
AP exchange	ACCEPT (user alice)	mutual authentication (ts+1 returned) = True

Attack scenario	Outcome
Replay captured authenticator, replay cache OFF	ACCEPT (user alice)
Replay captured authenticator, replay cache ON	REJECT: replay detected
Replay after skew window (timestamp check)	REJECT: clock skew > 300s
Fresh authenticator from alice (new timestamp)	ACCEPT (user alice)

Failure scenario	Outcome
Ticket within lifetime (service time = 1,000 s after issue)	ACCEPT (user alice)
Ticket expired (lifetime 3600 s, t = 9000 s)	REJECT: ticket expired
Client clock 700 s ahead of service	REJECT: clock skew > 300s

Scenario	Outcome
Stolen ticket used with a self-made authenticator (no session key)	REJECT: authenticator not encrypted under session key
New login (AS exchange) while KDC down	REJECT: KDC unavailable
Already-held service ticket used while KDC down	ACCEPT (user alice)

Offline guessing on the AS reply: an attacker who captures {K_c,tgs}K_bob can test password candidates offline because a correct key decrypts and authenticates the reply; with a six-word list the password "sunshine" was found after 5 tries. Mitigations: pre-authentication (the client first proves knowledge of its key with an encrypted timestamp), long random passwords, high iteration counts and account lockout. A stolen ticket alone is useless without the session key, but a stolen ticket together with its session key (for example copied from memory) lets the attacker impersonate the user until the ticket expires; a stolen TGT with the session key allows new service tickets.

Cross-understanding. (B) The client never sends its password: it uses the password only locally to derive K_alice and decrypt the AS reply, and the network carries only encrypted data. The service learns the user identity, the validity period and the session key, but not the password or K_alice. (A) The evidence is the replay table: with the replay cache off the captured authenticator is accepted a second time, with the cache on it is rejected as a replay, and after the skew window it is rejected by the timestamp check, while a fresh authenticator is accepted.

Q6 – ADVANCED SECURITY MINI-INVESTIGATIONS

Student A – Hash weakness and signature forgery

Collision versus second pre-image: in a collision attack the attacker chooses both messages (cost 2^(n/2)); in a second pre-image attack a message is fixed and the attacker must match its digest (cost 2^n). Collision attacks need the attacker to prepare both messages in advance, so they threaten signatures on documents or certificates that the attacker writes, but not signatures on arbitrary third-party messages.

Forgery experiment (RSA-512 signature over the first 32 bits of SHA-256): the attacker generates 131,072 benign variants "Pay Dr. Rao the sum of Rs 1,000 [ref i]" and tests malicious variants "Pay Dr. Mallory the sum of Rs 1,00,000 [ref j]" until a digest matches; after 1,408 malicious variants a collision was found in 0.1 s.

Item	Value
Benign message (signed)	Pay Dr. Rao the sum of Rs 1,000 [ref 7529]
Malicious message	Pay Dr. Mallory the sum of Rs 1,00,000 [ref 1407]
32-bit digest of benign	7a2f2e45
32-bit digest of malicious	7a2f2e45
Messages differ?	Yes
Signature on benign message verifies on malicious message?	Yes

Real-world impact: MD5 collisions were used to build a rogue CA certificate (2008) and the Flame malware forged Microsoft code-signing certificates (2012); a CA that signs a certificate whose body the attacker partly controls produces a signature that is valid for a second, malicious certificate. Therefore CAs must use collision-resistant hashes, randomise serial numbers and not rely on the hash alone. Digest length and algorithm choice decide the security level: the signature is only as strong as min(hash collision cost 2^(n/2), key strength). Hash policy: SHA-256 or stronger (SHA-384, SHA-512, SHA3) for all new signatures and certificates; MD5 and SHA-1 prohibited; at least 2048-bit RSA or 256-bit ECC; algorithm identifiers inside the signed data; periodic review (crypto-agility).

Student B verification: weakening the hash weakens the signature because the signature algorithm signs only the digest; if two messages share a digest, any signature on one is automatically valid on the other, regardless of how strong RSA or DSA is.

Student B – Kerberos versus X.509 and non-repudiation

Feature	Kerberos	X.509 / PKI
Key type	Symmetric (shared with KDC)	Asymmetric key pairs
Trusted third party	KDC (AS + TGS), online	CA hierarchy, trust anchors
Online dependence	KDC needed at login and for new tickets	Offline verification possible; revocation checks need online data
Scalability	Limited by KDC load and realm trust; cross-realm trust needs setup	Very scalable and global (web PKI)
Revocation	Short ticket lifetimes, disable account at KDC	CRL, OCSP, short-lived certificates
Typical environment	Enterprise or campus domains (single sign-on)	Web, e-mail, code and document signing, devices
Non-repudiation	No (shared secrets)	Yes, under the conditions of Q3

Kerberos gives no non-repudiation because every ticket and authenticator is protected by keys known to at least two parties (the KDC and the user or service), so any of them could have produced it; X.509 signatures use a private key held by one party, so a signature is attributable to that key holder. Single point of failure: the KDC (availability and confidentiality of every long-term key; the table in Q5 showed that new logins fail when the KDC is down). X.509 risk: compromise or mis-issuance by any trusted CA or intermediate lets an attacker obtain certificates for any name.

Users n	Pairwise shared keys n(n-1)/2	Kerberos (one key per principal with the KDC)	PKI (one key pair per principal; CA keys extra)
10	45	10	11 key pairs (incl. 1 CA)
100	4,950	100	101 key pairs (incl. 1 CA)
1,000	499,500	1,000	1,001 key pairs (incl. 1 CA)
10,000	49,995,000	10,000	10,001 key pairs (incl. 1 CA)

Recommendations: (i) campus single sign-on: Kerberos (or Kerberos-backed SSO), because it is fast, central account control and short-lived tickets suit a single administrative domain; (ii) legally binding electronic contracts: X.509-based digital signatures with timestamps, because only they provide non-repudiation and third-party verifiability.

Student A verification: a Kerberos ticket proves to the service, at that moment, that the KDC vouched for the user and that the sender knows the session key. A later third party cannot use it, because the service itself holds the key K_s and K_c,s and could have created the same ticket and authenticator, so the record does not prove that the user performed the action.

Signature scheme investigations (RSA, DSA/DSS, ElGamal, Schnorr)

ElGamal signature (toy p = 467, g = 2, private x = 127, public y = g^x = 132). Signing a hashed message h with nonce k (gcd(k, p-1) = 1): r = g^k mod p, s = (h - x r) k^-1 mod (p-1). Verification: y^r r^s = g^(x r) g^(k s) = g^(x r + h - x r) = g^h (mod p). Example (k = 213): (r, s) = (29, 51).

ElGamal test	Result
Valid message and signature	Accepted
Altered message ("900" instead of "100")	Rejected
Altered s	Rejected
Altered r	Rejected

ElGamal nonce reuse: two messages signed with the same k give identical r (check = Yes); the toy search recovers k (Yes), and then x = (h - k s) r^-1 mod (p-1) gives the private key 127. Poor randomness therefore destroys security even though the maths is unchanged.

Schnorr signature (same DSA-type group, p of 128 bits, q of 64 bits): commitment r = g^k, challenge e = H(r || m) mod q, response s = k + x e mod q. Verification: g^s y^-e = g^(k + x e - x e) = g^k = r, so recomputing e from r gives the same challenge. Tests: valid Accepted, altered message Rejected, altered response Rejected. The challenge must be unpredictable (hash of the commitment and message) otherwise a forger could choose e first and compute a matching r without knowing x; reusing k for two challenges gives s1 - s2 = x (e1 - e2), so x = (s1 - s2)(e1 - e2)^-1 mod q (private key recovered: Yes).

Property	RSA (hash-then-sign)	DSA / DSS	ElGamal	Schnorr
Key material	n = pq, e, d	p, q, g, x, y = g^x	p, g, x, y = g^x	p, q, g, x, y = g^x
Randomness	None per signature (with deterministic padding such as PSS salt optional)	Fresh secret k each signature	Fresh secret k, gcd(k, p-1) = 1	Fresh secret k each signature
Verification	s^e = H(m) (1 exponentiation)	Two exponentiations, v = r	y^r r^s = g^h	g^s y^-e recomputes challenge
Forgery resistance	Factoring plus a secure hash and padding	Discrete log plus a secure hash	Discrete log; needs a hash (no hash allows forgeries)	Discrete log in the random-oracle model, with proof
Effect of nonce reuse	Not applicable	Private key recovered (shown in Q3)	Private key recovered (shown above)	Private key recovered (shown above)
Non-repudiation	Yes, if the key is exclusive and certified	Same	Same	Same

Q7 – DESIGN A SECURE AUTHENTICATION AND INTEGRITY SCHEME FROM FIRST PRINCIPLES

Scenario: signed over-the-air firmware updates for infusion pumps (limited memory, often offline, unreliable clocks).

Stage 1 – Independent proposals (retained)

Aspect	Student A initial proposal	Student B initial proposal
Goals and primitives	Manufacturer signs SHA-256 of the image with ECDSA; the pump holds the manufacturer public key and verifies the signature (integrity, authenticity, non-repudiation).	Pump and manufacturer share an HMAC key per device; the update carries HMAC-SHA-256(image) and a timestamp; the pump checks tag and time (integrity, authenticity).
Key material	Pump: one embedded public key. Manufacturer: one private signing key.	Pump: one secret key per device (factory programmed). Manufacturer: database of all keys.
Message flow	Server sends image, signature; pump hashes and verifies.	Server sends image, timestamp and tag; pump recomputes HMAC and checks the timestamp is recent.
Known gaps	No rollback protection, no key rotation or revocation, single key compromise breaks every pump.	No non-repudiation, key database is a single secret target, relies on the pump clock which is unreliable.

Stage 2 – Comparison of the two designs

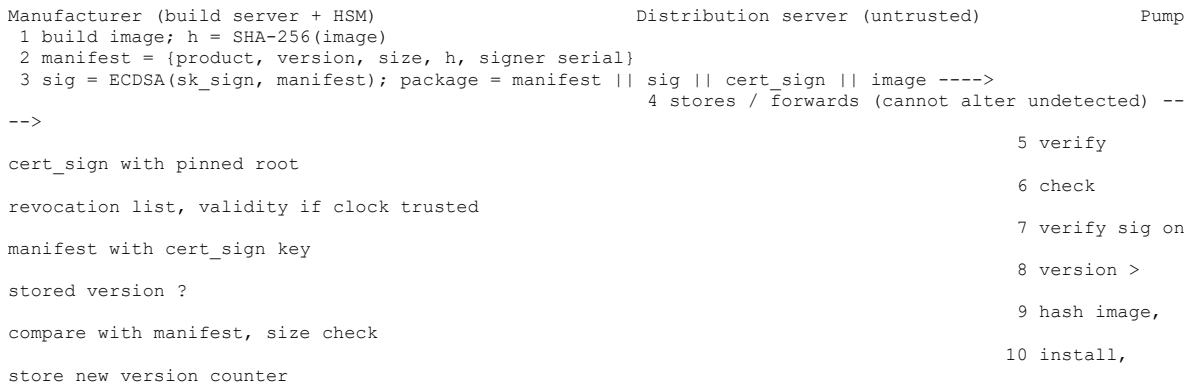
Question	Record
Agreement	Both hash the image with SHA-256 and verify before installing.
Difference	A uses asymmetric signature, B uses symmetric MAC and clock time.
Strengths	A: non-repudiation, one public key on the device, no secret on the pump. B: cheap verification and small metadata.
Weaknesses	A: no rollback, revocation or key hierarchy. B: no non-repudiation, secrets on every device, depends on a correct clock.
Missing threats	Rollback to an old vulnerable version, stolen signing key, offline pump, wrong clock, revoked key.
Retained ideas	From A: signatures and a pinned public key. From B: a freshness field, replaced by a monotonic version counter because clocks are unreliable. Added: certificate hierarchy, revocation list, manifest.

Stage 3 – Joint final design

Threat model. The attacker controls the network and the distribution server, can modify, replay and delay updates, can serve old signed versions, may steal a device and read its memory, and may obtain a signing key by compromise. The attacker cannot break SHA-256 or ECDSA P-256 and cannot change the pump's protected pinned root key.

Goal	Mechanism
Integrity	SHA-256 digest of the image inside the signed manifest
Authentication of the manufacturer	ECDSA signature by a signing key certified by the manufacturer root (pinned in the pump)
Non-repudiation	Asymmetric signature plus certificate, logged signing events and HSM-held private key
Freshness / no rollback	Monotonic version counter in the manifest compared with the counter stored in the pump (no clock needed)
Revocation	Revocation list (serial numbers) carried inside newer signed manifests and stored in the pump; short signing-certificate lifetime when a trusted clock is available
Pump-to-server authentication	Pump device certificate or Kerberos-style ticket with nonce challenge for service sessions; not needed for update verification

Key and certificate management. The root key is kept offline in an HSM and used only to certify signing keys; the signing key is in an online HSM with dual-control access and an audit log. Signing certificates are issued yearly, rotated before expiry, and revoked by adding the serial to the revocation list in the next update. The pump stores only the root certificate (pinned in protected memory), the last accepted version and the revocation list. If a signing key is compromised: revoke it, issue a new one, ship a new signed update, and use the version counter so that old malicious updates cannot be rolled forward.



Offline pump or wrong clock. Verification needs no network: the pinned root, the carried certificate and the manifest signature are sufficient. If the clock is not trusted, the pump skips certificate-validity checks (shown in the tests) and relies on the version counter and the revocation list; when it reconnects it takes a time value signed by the manufacturer and updates its trusted time.

```
SIGN(image, version):
  h = SHA256(image)
  M = {product, version, size, h, serial}
  sig = ECDSA_sign(sk_sign, M)
  return M || sig || cert_sign || image
```

```
VERIFY(package, stored_version, revoked, now_or_none):
  c = parse(package.cert)
  if not verify(root_pub, c.tbs, c.sig): REJECT
  if c.serial in revoked: REJECT
  if now_or_none is trusted and not (c.nbf <= now <=
c.naf): REJECT
  if not ECDSA_verify(c.pub, M, sig): REJECT
  if M.version <= stored_version: REJECT      (rollback)
  if SHA256(image) != M.h or len(image) != M.size: REJECT
  install; stored_version = M.version; ACCEPT
```

Stage 4 – Student A: security argument

Attack	What it needs	Effort / limit	Prevented by
Modify the image	Second pre-image of SHA-256	about 2^256 (collision 2^128; Q1 shows effort 2^(k/2): k = 32 took 88,937 hashes, about 2^16.4)	Digest in the signed manifest
Forge the manifest signature	Break ECDSA P-256 or obtain the key	about 2^128 operations; nonce reuse would leak the key (Q3), so signing in an HSM with a secure nonce	Signature and HSM
Rollback to an old version	Replay an old signed package	Needs no cryptographic effort	Version counter in the manifest
Stolen signing key	Compromise of the key store	Operational, not cryptographic	HSM, rotation, revocation list, short certificate lifetime
Rogue certificate	Create a CA-signed certificate	Needs compromise of the root key	Pinned offline root, pathLen/EKU checks
Weak hash	Collision as in MD5/SHA-1 (Q6)	Not applicable with SHA-256 (2^128)	Hash policy: SHA-256 or stronger

Residual risks: compromise of the root or the signing HSM, implementation bugs in the verifier, physical attacks on the pump memory, a bad clock if the pump trusts it for validity, supply-chain compromise before signing, and denial of service by dropping updates.

Stage 4 – Student B: experimental validation

The final verification was implemented (class Pump, Appendix B) with ECDSA P-256 from the library, and compared with a baseline that publishes a bare SHA-256 digest beside a 1 MiB image.

Test case	Final design	Baseline (bare SHA-256)
Valid update v5	ACCEPT	ACCEPT
Modified image (attacker keeps old manifest)	REJECT: image hash mismatch	REJECT
Modified image AND digest recomputed	REJECT: image hash mismatch	ACCEPT
Modified signature	REJECT: manifest signature invalid	not applicable (no signature)
Replay of older signed version v3 (rollback)	REJECT: rollback / replay (version 3 <= installed 4)	ACCEPT (old image + old digest are consistent)
Wrong key (manifest signed by attacker key)	REJECT: manifest signature invalid	not applicable
Attacker certificate (not issued by pinned root)	REJECT: signing certificate not issued by pinned root	not applicable
Revoked signing key (serial on pump revocation list)	REJECT: signing certificate revoked	not applicable
Expired signing certificate (trusted clock)	REJECT: signing certificate outside validity (trusted clock)	not applicable
Pump clock untrusted / wrong: validity check skipped, version counter still enforced	ACCEPT ; old v3: REJECT: rollback / replay (version 3 <= installed 4)	not applicable

Measure (1 MiB image)	Final design	Baseline
Hash operations (SHA-256 compression blocks)	16,389 (image 16,385 + manifest 4)	16,385
Signature verifications	2 (certificate and manifest)	0
Added metadata (bytes)	624: manifest 197 + signature 72 + certificate 355	32 (digest)
Verification time (ms, this machine)	1.07	0.67

The baseline fails the attacks that matter: an attacker who replaces image and digest is accepted, and a consistent old image with its old digest (rollback) is accepted; it also gives no authenticity, no revocation and no non-repudiation. The cost of the final design is only two signature verifications, about 0.3 ms of extra time and 622 bytes of metadata per update.

Stage 5 – Cross-defence

Student A (experimental evidence): the table of test cases shows that every designed attack (modified image, modified signature, wrong key, attacker certificate, rollback, revoked key, expired certificate) is rejected by the step intended for it, and that the baseline accepts the two most important ones; the collision experiments of Q1 confirm the 2^(n/2) argument used for the digest size.

Student B (security argument): integrity comes from the digest in the manifest, the manifest is authenticated by the signature, and the signing key is authenticated by the pinned root through the certificate; rollback protection comes from the version counter; revocation from the list; and non-repudiation from the exclusive private key plus the certificate chain, so each mechanism contributes exactly one link in the argument.

Stage 6 – Joint evaluation

Property	Provided by
Integrity	SHA-256 digest in the signed manifest
Authentication	ECDSA signature and certificate chain to the pinned root
Non-repudiation	Private key held only in the manufacturer HSM, certificate, signed manifest and audit log (limited by key-compromise claims)

Costs: about 2 signature verifications and one image hash per update, 622 bytes of extra data, one pinned root and a small revocation list on the pump, and operational cost of HSMs, rotation and revocation. Failure conditions: compromise of the root or signing key, a flawed verifier or secure-boot implementation, a bad clock if validity is trusted, or a revoked key whose revocation list never reached the pump. When simpler mechanisms are better: HMAC with a per-device key is appropriate for authenticating a pump to the hospital server inside one trust domain where non-repudiation is not needed, and Kerberos-style tickets suit session authentication to the central server; neither replaces a signature for firmware, because the manufacturer must not be able to deny, and the pump must not hold a key that could forge updates.

APPENDIX A – SOURCE CODE (crypto3.py: hashes, MD hash, HMAC, RSA, DSA, ElGamal, Schnorr)

```
# crypto3.py - student-implemented core for CO3 (educational only, toy parameters)
import hashlib, os, random, math
SR=random.SystemRandom()
# ---------- number theory
def egcd(a,b):
    if b==0: return a,1,0
    g,x,y=egcd(b,a%b); return g,y,x-(a//b)*y
def inv(a,m):
    g,x,_=egcd(a%m,m)
    if g!=1: raise ValueError('no inverse')
    return x%m
def mr(n,rounds=20):
    if n<4: return n in(2,3)
    if n%2==0: return False
    d,s=n-1,0
    while d%2==0: d//=2; s+=1
    for _ in range(rounds):
        a=SR.randrange(2,n-1); x=pow(a,d,n)
        if x in(1,n-1): continue
        for _ in range(s-1):
            x=x*x%n
            if x==n-1: break
        else: return False
    return True
def gen_prime(bits):
    while True:
        c=SR.getrandbits(bits)|(1<<(bits-1))|1
        if mr(c): return c
# ---------- Q1: toy hashes and collision search
def bytesum(m): return sum(m)&0xFFFFFFFF
def xorfold(m):
    m=m+b'\0'*(-len(m)%4); r=0
    for i in range(0,len(m),4): r^=int.from_bytes(m[i:i+4],'big')
    return r
def sha(m): return int.from_bytes(hashlib.sha256(m).digest(),'big')
def sha32(m): return int.from_bytes(hashlib.sha256(m).digest()[:4],'big')
def flipbit(m,i): b=bytearray(m); b[i//8]^=1<<(i%8); return bytes(b)
def trunc(m,k): return int.from_bytes(hashlib.sha256(m).digest()[:8],'big')>>(64-k)
def find_collision(k):
    seen={}; pre=os.urandom(8); n=0
    while True:
        m=pre+n.to_bytes(8,'big'); h=trunc(m,k); n+=1
        if h in seen: return n,seen[h],m
        seen[h]=m
def find_preimage(k,target):
    pre=os.urandom(8); n=0
    while True:
        n+=1
        if trunc(pre+n.to_bytes(8,'big'),k)==target: return n
# ---------- Q2: toy Merkle-Damgard hash (64-bit chaining value, 16-byte block)
M64=(1<<64)-1; IV=0x0123456789ABCDEF; BLK=16
def compress(s,blk):
    for i in(0,8):
        s^=int.from_bytes(blk[i:i+8],'big'); s=(s*0x9E3779B97F4A7C15)&M64
        s^=s>>29; s=((s<<13)|(s>>51))&M64
    return s
def md_pad(total_len): return b'\x80'+b'\0'*((-(total_len+9))%BLK)+(total_len*8).to_bytes(8,'big')
def md_state(msg,iv=IV,offset=0):
    data=msg+md_pad(len(msg)+offset); s=iv
    for i in range(0,len(data),BLK): s=compress(s,data[i:i+BLK])
    return s
def md(msg,iv=IV,offset=0): return md_state(msg,iv,offset).to_bytes(8,'big')
def naive_mac(key,msg): return md(key+msg)
def hmac_generic(h,B,key,msg):
    if len(key)>B: key=h(key)
    key=key+b'\0'*(B-len(key))
    return h(bytes(x^0x5c for x in key)+h(bytes(x^0x36 for x in key)+msg))
def length_extend(tag,klen,msg,ext):
    glue=md_pad(klen+len(msg)); forged=msg+glue+ext
    ftag=md(ext,iv=int.from_bytes(tag,'big'),offset=klen+len(msg)+len(glue))
    return forged,ftag
# ---------- Q3: RSA hash-then-sign, DSA, ElGamal, Schnorr
def rsa_keygen(bits,e=65537):
    while True:
        p,q=gen_prime(bits//2),gen_prime(bits//2)
        if p!=q and math.gcd(e,(p-1)*(q-1))==1: return (p*q,e),(p*q,inv(e,(p-1)*(q-1)))
def H(m): return int.from_bytes(hashlib.sha256(m).digest(),'big')
def rsa_sign(priv,m): return pow(H(m)%priv[0],priv[1],priv[0])
def rsa_verify(pub,m,s): return pow(s,pub[1],pub[0])==H(m)%pub[0]
def dsa_params(qbits=64,pbits=128):
    q=gen_prime(qbits)
    while True:
        k=SR.getrandbits(pbits-qbits)|(1<<(pbits-qbits-1)); p=k*q+1
```

```
        if p.bit_length()==pbits and mr(p): break
    while True:
        h=SR.randrange(2,p-1); g=pow(h,(p-1)//q,p)
        if g>1: return p,q,g
def dsa_sign(P,x,m,k=None):
    p,q,g=P
    while True:
        kk=k or SR.randrange(2,q-1); r=pow(g,kk,p)%q
        s=inv(kk,q)*(H(m)%q+x*r)%q
        if r and s: return r,s
def dsa_verify(P,y,m,sig):
    p,q,g=P; r,s=sig
    if not(0<r<q and 0<s<q): return False
    w=inv(s,q); u1=H(m)%q*w%q; u2=r*w%q
    return pow(g,u1,p)*pow(y,u2,p)%p%q==r
def eg_sign(p,g,x,h,k): r=pow(g,k,p); return r,(h-x*r)*inv(k,p-1)%(p-1)
def eg_verify(p,g,y,h,sig): r,s=sig; return 0<r<p and pow(y,r,p)*pow(r,s,p)%p==pow(g,h,p)
def schnorr_sign(P,x,m,k=None):
    p,q,g=P; k=k or SR.randrange(2,q-1); r=pow(g,k,p)
    e=H(r.to_bytes(32,'big')+m)%q; return e,(k+x*e)%q
def schnorr_verify(P,y,m,sig):
    p,q,g=P; e,s=sig; r=pow(g,s,p)*pow(y,q-e,p)%p
    return H(r.to_bytes(32,'big')+m)%q==e
```

APPENDIX B – SOURCE CODE (pki_krb_fw.py: X.509 validation, Kerberos simulation, firmware update)

```
# pki_krb_fw.py - Q4 chain validation, Q5 Kerberos simulation, Q7 firmware-update scheme (educational)
import datetime as dt, json, os, hashlib, time
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.exceptions import InvalidSignature
D=dt.datetime; UTC=dt.timezone.utc
NOW=D(2026,11,6,tzinfo=UTC)
# ================== Q4
def name(cn): return x509.Name([x509.NameAttribute(NameOID.COMMON_NAME,cn)])
def mk(cn,key,issuer_cn,issuer_key,ca,nb,na,san=None,serial=None,pathlen=None,keycert=None):

b=(x509.CertificateBuilder().subject_name(name(cn)).issuer_name(name(issuer_cn)).public_key(key.public_ke
y())
       .serial_number(serial or x509.random_serial_number()).not_valid_before(nb).not_valid_after(na)
       .add_extension(x509.BasicConstraints(ca=ca,path_length=pathlen),critical=True))
    ku=ca if keycert is None else keycert
    b=b.add_extension(x509.KeyUsage(digital_signature=not
ku,key_cert_sign=ku,crl_sign=ku,content_commitment=False,key_encipherment=False,data_encipherment=False,k
ey_agreement=False,encipher_only=False,decipher_only=False),critical=True)
    if san: b=b.add_extension(x509.SubjectAlternativeName([x509.DNSName(san)]),critical=False)
    return b.sign(issuer_key,hashes.SHA256())
def validate(chain,anchors,host,now,revoked=frozenset(),check_rev=True):
    """chain=[leaf, intermediate,...]; returns (accepted, failing_step)"""
    trusted={c.subject.rfc4514_string():c for c in anchors}
    full=list(chain)
    top=full[-1]
    if top.issuer.rfc4514_string() not in trusted: return False,'S1 trust anchor: issuer of top
certificate is not a trusted root'
    full.append(trusted[top.issuer.rfc4514_string()])
    for i,c in enumerate(full):
        if not(c.not_valid_before_utc<=now<=c.not_valid_after_utc): return False,'S4 validity period
(cert %d: %s)'%(i,c.subject.rfc4514_string())
    for i in range(len(full)-1):
        c,iss=full[i],full[i+1]
        if c.issuer!=iss.subject: return False,'S2 issuer/subject name match (cert %d)'%i
        try:
iss.public_key().verify(c.signature,c.tbs_certificate_bytes,ec.ECDSA(c.signature_hash_algorithm))
        except InvalidSignature: return False,'S3 signature verification (cert %d)'%i
    try: full[-1].public_key().verify(full[-1].signature,full[-1].tbs_certificate_bytes,ec.ECDSA(full[-
1].signature_hash_algorithm))
    except InvalidSignature: return False,'S3 root self-signature'
    for i,iss in enumerate(full[1:],1):
        bc=iss.extensions.get_extension_for_class(x509.BasicConstraints).value
        if not bc.ca: return False,'S5 basicConstraints CA flag (cert %d is not a CA)'%i
        ku=iss.extensions.get_extension_for_class(x509.KeyUsage).value
        if not ku.key_cert_sign: return False,'S6 keyUsage keyCertSign (cert %d)'%i
    try:
sans=full[0].extensions.get_extension_for_class(x509.SubjectAlternativeName).value.get_values_for_type(x5
09.DNSName)
    except x509.ExtensionNotFound: sans=[]
    if host not in sans: return False,'S7 hostname match (SAN=%s)'%sans
    if check_rev:
        for c in full[:-1]:
            if c.serial_number in revoked: return False,'S8 revocation (serial %x
```

```
listed)'%(c.serial_number&0xFFFF)
    return True,'accepted'
def q4():
    out={}
    rk,ik,lk,rogue=[ec.generate_private_key(ec.SECP256R1()) for _ in range(4)]
    y=dt.timedelta(days=365); host='clinic.example'
    root=mk('Root CA',rk,'Root CA',rk,True,NOW-5*y,NOW+10*y,pathlen=1)
    inter=mk('Intermediate CA',ik,'Root CA',rk,True,NOW-2*y,NOW+5*y,pathlen=0)
    leaf=mk(host,lk,'Intermediate CA',ik,False,NOW-30*dt.timedelta(days=1),NOW+y,san=host)

out['structure']=[[c.subject.rfc4514_string(),c.issuer.rfc4514_string(),'%x'%(c.serial_number&0xFFFFFF),c
.not_valid_before_utc.date().isoformat()+' to
'+c.not_valid_after_utc.date().isoformat(),c.extensions.get_extension_for_class(x509.BasicConstraints).va
lue.ca] for c in(root,inter,leaf)]
    cases=[]
    def run(label,chain,anchors=[root],h=host,now=NOW,rev=frozenset(),chk=True):
cases.append([label]+list(validate(chain,anchors,h,now,rev,chk)))
    run('Valid chain',[leaf,inter])
    run('Expired end-entity certificate',[mk(host,lk,'Intermediate CA',ik,False,NOW-3*y,NOW-
y,san=host),inter])
    run('Not-yet-valid end-entity certificate',[mk(host,lk,'Intermediate
CA',ik,False,NOW+y,NOW+2*y,san=host),inter])
    run('Wrong issuer (signed by rogue CA, name Rogue CA)',[mk(host,lk,'Rogue CA',rogue,False,NOW-
y,NOW+y,san=host),inter])
    run('Right issuer name but signed with rogue key',[mk(host,lk,'Intermediate CA',rogue,False,NOW-
y,NOW+y,san=host),inter])
    der=bytearray(leaf.public_bytes(serialization.Encoding.DER)); i=der.index(b'clinic.example');
der[i+5]=ord('x')
    run('Tampered field (certificate bytes modified after
signing)',[x509.load_der_x509_certificate(bytes(der)),inter],h='clinix.example')
    run('Hostname mismatch (valid chain, wrong name requested)',[leaf,inter],h='pharmacy.example')
    sub=mk('Fake sub-CA',ec.generate_private_key(ec.SECP256R1()),host,lk,True,NOW-y,NOW+y)  # issued by
an end-entity key
    fake_leaf=mk('evil.example',rogue,'Fake sub-CA',ec.generate_private_key(ec.SECP256R1()),False,NOW-
y,NOW+y,san='evil.example')
    run('End-entity certificate used as a CA',[mk('evil.example',rogue,host,lk,False,NOW-
y,NOW+y,san='evil.example'),leaf,inter],h='evil.example')
    run('Untrusted root (anchor not installed)',[leaf,inter],anchors=[mk('Other Root',rk,'Other
Root',rk,True,NOW-y,NOW+y,pathlen=1)])
    run('Revoked end-entity, revocation checked',[leaf,inter],rev=frozenset({leaf.serial_number}))
    run('Revoked end-entity, revocation NOT checked (fail-
open)',[leaf,inter],rev=frozenset({leaf.serial_number}),chk=False)
    out['cases']=cases
    return out
# ================ Q5 Kerberos
SKEW=300
def kdf(pw,salt): return PBKDF2HMAC(hashes.SHA256(),32,salt,2000).derive(pw.encode())
def seal(k,obj): n=os.urandom(12); return n+AESGCM(k).encrypt(n,json.dumps(obj).encode(),None)
def unseal(k,b): return json.loads(AESGCM(k).decrypt(b[:12],b[12:],None))
class KDC:
    def __init__(s,users):
        s.up=True; s.k_tgs=os.urandom(32); s.svc={'files':os.urandom(32)}
        s.users={u:kdf(p,b'REALM'+u.encode()) for u,p in users.items()}
    def as_req(s,user,nonce,now):
        if not s.up: raise ConnectionError('KDC unavailable')
        sk=os.urandom(32).hex(); tgt=seal(s.k_tgs,{'u':user,'k':sk,'start':now,'life':36000})
        return seal(s.users[user],{'k':sk,'nonce':nonce,'life':36000}),tgt           # AS-REP
    def tgs_req(s,tgt,auth,svc,now):
        if not s.up: raise ConnectionError('KDC unavailable')
        t=unseal(s.k_tgs,tgt); a=unseal(bytes.fromhex(t['k']),auth)
        if a['u']!=t['u'] or abs(a['ts']-now)>SKEW: raise PermissionError('TGS: bad authenticator')
        if now>t['start']+t['life']: raise PermissionError('TGS: TGT expired')
        sk=os.urandom(32).hex(); tk=seal(s.svc[svc],{'u':t['u'],'k':sk,'start':now,'life':3600})
        return seal(bytes.fromhex(t['k']),{'k':sk,'svc':svc}),tk
class Service:
    def __init__(s,key,replay=True): s.key=key; s.replay=replay; s.cache=set()
    def ap_req(s,tk,auth,now):
        t=unseal(s.key,tk)
        if now>t['start']+t['life']: return 'REJECT: ticket expired',None
        k=bytes.fromhex(t['k'])
        try: a=unseal(k,auth)
        except Exception: return 'REJECT: authenticator not encrypted under session key',None
        if a['u']!=t['u']: return 'REJECT: user mismatch',None
        if abs(a['ts']-now)>SKEW: return 'REJECT: clock skew > %ds'%SKEW,None
        if s.replay:
            if (a['u'],a['ts']) in s.cache: return 'REJECT: replay detected',None
            s.cache.add((a['u'],a['ts']))
        return 'ACCEPT (user %s)'%t['u'],seal(k,{'ts':a['ts']+1})                    # AP-REP (mutual
auth)
def q5():
    out={}; K=KDC({'alice':'Tr0ub4dor','bob':'sunshine'}); svc=Service(K.svc['files']); t0=1_000_000
    rep,tgt=K.as_req('alice',77,t0); kc=kdf('Tr0ub4dor',b'REALMalice'); r=unseal(kc,rep);
ktgs=bytes.fromhex(r['k'])
    a1=seal(ktgs,{'u':'alice','ts':t0+1}); srep,tk=K.tgs_req(tgt,a1,'files',t0+1);
ks=bytes.fromhex(unseal(ktgs,srep)['k'])
    auth=seal(ks,{'u':'alice','ts':t0+2}); res,aprep=svc.ap_req(tk,auth,t0+2)
    mut=unseal(ks,aprep)['ts']==t0+3
    out['normal']=[['AS exchange','alice sends name + nonce; receives TGT and {K_c,tgs,
```

```
nonce}K_alice','nonce echoed = %s'%(r['nonce']==77)],['TGS exchange','TGT +
authenticator{K_c,tgs}','service ticket issued'],['AP exchange',res,'mutual authentication (ts+1
returned) = %s'%mut]]
    # replay with protection
    cap_auth=auth; res2,_=svc.ap_req(tk,cap_auth,t0+5);
    svc2=Service(K.svc['files'],replay=False); svc2.ap_req(tk,auth,t0+2);
res3,_=svc2.ap_req(tk,cap_auth,t0+5)
    res4,_=svc.ap_req(tk,cap_auth,t0+2+SKEW+60)
    # fresh authenticator by legit user is accepted
    res5,_=svc.ap_req(tk,seal(ks,{'u':'alice','ts':t0+9}),t0+9)
    out['attacks']=[['Replay captured authenticator, replay cache OFF',res3],['Replay captured
authenticator, replay cache ON',res2],['Replay after skew window (timestamp check)',res4],['Fresh
authenticator from alice (new timestamp)',res5]]
    res6,_=svc.ap_req(tk,seal(ks,{'u':'alice','ts':t0+1000}),t0+1000);
res7,_=svc.ap_req(tk,seal(ks,{'u':'alice','ts':t0+9000}),t0+9000)
    res8,_=svc.ap_req(tk,seal(ks,{'u':'alice','ts':t0+2+700}),t0+2)       # client clock 700 s ahead
    out['failures']=[['Ticket within lifetime (service time = 1,000 s after issue)',res6],['Ticket
expired (lifetime 3600 s, t = 9000 s)',res7],['Client clock 700 s ahead of service',res8]]
    # stolen ticket without session key
    res9,_=svc.ap_req(tk,seal(os.urandom(32),{'u':'alice','ts':t0+20}),t0+20)
    out['stolen']=res9
    # KDC down
    K.up=False
    try: K.as_req('bob',1,t0); a='no error'
    except ConnectionError as e: a='REJECT: '+str(e)
    cached,_=Service(K.svc['files'],True).ap_req(tk,seal(ks,{'u':'alice','ts':t0+30}),t0+30)
    out['kdc']=[['New login (AS exchange) while KDC down',a],['Already-held service ticket used while KDC
down',cached]]
    K.up=True
    # offline guessing on AS reply
    rep_b,_=K.as_req('bob',5,t0); words=['password','123456','letmein','qwerty','sunshine','dragon'];
tries=0; found=None
    for w in words:
        tries+=1
        try: unseal(kdf(w,b'REALMbob'),rep_b); found=w; break
        except Exception: pass
    out['offline']=[found,tries,len(words)]
    return out
# ================ Q7 firmware
def sha_blocks(n): return -(-(n+9)//64)
def fw_build():
    root=ec.generate_private_key(ec.SECP256R1()); sk=ec.generate_private_key(ec.SECP256R1());
y=dt.timedelta(days=365)
    rootc=mk('MedDev Root',root,'MedDev Root',root,True,NOW-5*y,NOW+10*y,pathlen=1)
    signc=mk('MedDev Firmware Signing 2026',sk,'MedDev Root',root,False,NOW-
30*dt.timedelta(days=1),NOW+y,keycert=False)
    return root,sk,rootc,signc
def make_update(sk,signc,version,image,epoch=1):
    man=json.dumps({'product':'pump-
x1','version':version,'epoch':epoch,'size':len(image),'sha256':hashlib.sha256(image).hexdigest(),'signer'
:'%x'%signc.serial_number},sort_keys=True).encode()
    return
{'manifest':man,'sig':sk.sign(man,ec.ECDSA(hashes.SHA256())),'cert':signc.public_bytes(serialization.Enco
ding.DER),'image':image}
class Pump:
    def __init__(s,rootc,version,revoked=frozenset(),clock=None): s.root=rootc; s.version=version;
s.revoked=revoked; s.clock=clock; s.cost={'hash_blocks':0,'sig_verify':0}
    def verify(s,u):
        s.cost={'hash_blocks':0,'sig_verify':0}
        c=x509.load_der_x509_certificate(u['cert'])
        try:
s.root.public_key().verify(c.signature,c.tbs_certificate_bytes,ec.ECDSA(c.signature_hash_algorithm));
s.cost['sig_verify']+=1
        except InvalidSignature: return 'REJECT: signing certificate not issued by pinned root'
        if c.serial_number in s.revoked: return 'REJECT: signing certificate revoked'
        if s.clock is not None and not(c.not_valid_before_utc<=s.clock<=c.not_valid_after_utc): return
'REJECT: signing certificate outside validity (trusted clock)'
        try: c.public_key().verify(u['sig'],u['manifest'],ec.ECDSA(hashes.SHA256()));
s.cost['sig_verify']+=1
        except InvalidSignature: return 'REJECT: manifest signature invalid'
        m=json.loads(u['manifest']); s.cost['hash_blocks']+=sha_blocks(len(u['manifest']))
        if m['version']<=s.version: return 'REJECT: rollback / replay (version %d <= installed
%d)'%(m['version'],s.version)
        s.cost['hash_blocks']+=sha_blocks(len(u['image']))
        if hashlib.sha256(u['image']).hexdigest()!=m['sha256'] or len(u['image'])!=m['size']: return
'REJECT: image hash mismatch'
        return 'ACCEPT'
def q7():
    out={}; root,sk,rootc,signc=fw_build(); img=os.urandom(1<<20)
    good=make_update(sk,signc,5,img); pump=Pump(rootc,4,clock=NOW)
    cases=[]; base_digest=hashlib.sha256(img).hexdigest()
    def base(image,digest): return 'ACCEPT' if hashlib.sha256(image).hexdigest()==digest else 'REJECT'
    cases.append(['Valid update v5',pump.verify(good),base(img,base_digest)])
    cost=dict(pump.cost)
    bad=bytearray(img); bad[1000]^=1; u=dict(good); u['image']=bytes(bad)
    cases.append(['Modified image (attacker keeps old
manifest)',pump.verify(u),base(bytes(bad),base_digest)])
    bad_digest=hashlib.sha256(bytes(bad)).hexdigest()
```

```
    cases.append(['Modified image AND digest recomputed',pump.verify(u),base(bytes(bad),bad_digest)])
    s=bytearray(good['sig']); s[10]^=1; u=dict(good); u['sig']=bytes(s)
    cases.append(['Modified signature',pump.verify(u),'not applicable (no signature)'])
    old=make_update(sk,signc,3,os.urandom(2048)); cases.append(['Replay of older signed version v3
(rollback)',pump.verify(old),'ACCEPT (old image + old digest are consistent)'])
    ek=ec.generate_private_key(ec.SECP256R1()); ev=make_update(ek,signc,6,img); cases.append(['Wrong key
(manifest signed by attacker key)',pump.verify(ev),'not applicable'])
    rogue_cert=mk('MedDev Firmware Signing 2026',ek,'MedDev Root',ek,False,NOW-
dt.timedelta(days=30),NOW+dt.timedelta(days=365),keycert=False)
    ru=make_update(ek,rogue_cert,6,img); cases.append(['Attacker certificate (not issued by pinned
root)',pump.verify(ru),'not applicable'])
    p2=Pump(rootc,4,revoked=frozenset({signc.serial_number}),clock=NOW); cases.append(['Revoked signing
key (serial on pump revocation list)',p2.verify(good),'not applicable'])
    p3=Pump(rootc,4,clock=NOW+dt.timedelta(days=800)); cases.append(['Expired signing certificate
(trusted clock)',p3.verify(good),'not applicable'])
    p4=Pump(rootc,4,clock=None); cases.append(['Pump clock untrusted / wrong: validity check skipped,
version counter still enforced',p4.verify(good)+' ; old v3: '+p4.verify(old),'not applicable'])
    out['cases']=cases

out['cost']=[len(good['manifest']),len(good['sig']),len(good['cert']),len(good['manifest'])+len(good['sig
'])+len(good['cert']),cost['hash_blocks'],cost['sig_verify'],len(img),sha_blocks(len(img))]
    t=time.perf_counter(); [Pump(rootc,4,clock=NOW).verify(good) for _ in range(20)];
out['time_ms']=round((time.perf_counter()-t)/20*1000,2)
    t=time.perf_counter(); [hashlib.sha256(img).hexdigest() for _ in range(20)];
out['base_ms']=round((time.perf_counter()-t)/20*1000,2)
    out['base_meta']=64
    return out
if __name__=='__main__':
    R={'q4':q4(),'q5':q5(),'q7':q7()}; json.dump(R,open('results3b.json','w'),indent=1,default=str);
print(json.dumps(R,indent=1,default=str))
```

APPENDIX C – EXPERIMENT DRIVER (run3.py)

```
import json,statistics,time,os,math,hmac,hashlib,random
from crypto3 import *
R={}; random.seed(3); T=time.perf_counter
# ===== Q1 A: avalanche
hf={'Byte-sum (32-bit)':(bytesum,32),'XOR-fold (32-bit)':(xorfold,32),'SHA-256 truncated to 32
bits':(sha32,32),'SHA-256 (256-bit)':(sha,256)}
av=[]
for size in(16,64,256,1024):
    for name,(f,n) in hf.items():
        ch=[]
        for _ in range(1000):
            m=os.urandom(size); h0=f(m); i=random.randrange(size*8);
ch.append(bin(h0^f(flipbit(m,i))).count('1'))

av.append([name,size,1000,round(statistics.mean(ch),2),round(n/2,1),round(statistics.pstdev(ch),2),round(
math.sqrt(n)/2,2)])
R['q1_av']=av
# toy hash second preimage / collision
m=b'PAY 100 TO DR A'; m2=bytearray(m); m2[0],m2[1]=m2[1],m2[0]
R['q1_sp']=[m.decode(),bytes(m2).decode(),bytesum(m),bytesum(bytes(m2)),xorfold(m),xorfold(bytes(m2)),sha
(m)>>224,sha(bytes(m2))>>224]
m3=bytearray(m); m3[2]+=1; m3[3]-=1
R['q1_sp2']=[bytes(m3).decode(),bytesum(bytes(m3))==bytesum(m)]
# ===== Q1 B: birthday
bd=[]; raw=[]
for k in(16,20,24,28,32):
    tr=20 if k<=28 else 10; ns=[find_collision(k)[0] for _ in range(tr)]; raw.append([k,ns])

bd.append([k,tr,min(ns),int(statistics.mean(ns)),max(ns),round(1.2533*2**(k/2)),round(2**(k/2)),round(sta
tistics.mean(ns)/2**(k/2),2)])
R['q1_bd']=bd; R['q1_bd_raw']=raw
pre=[]
for k in(8,12,16):
    ns=[find_preimage(k,random.getrandbits(k)) for _ in range(20)];
pre.append([k,int(statistics.mean(ns)),2**k])
R['q1_pre']=pre
# ===== Q2
key=b'K3Y-SECRET'; msg=b'amount=100&to=alice'
t=md(msg); R['q2_digest']=[t.hex(),md(msg+b'!').hex()]
# attack 1: plain hash, attacker changes file and digest
evil=b'amount=900&to=mallory'; R['q2_plain']=[md(msg).hex(),md(evil).hex(),md(evil)==md(evil)]
tag=naive_mac(key,msg); ext=b'&to=mallory'
forged,ftag=length_extend(tag,len(key),msg,ext)
R['q2_le']=[tag.hex(),repr(forged),ftag.hex(),naive_mac(key,forged).hex(),naive_mac(key,forged)==ftag,for
ged.hex()]
# wrong key length guess
R['q2_le_guess']=[[kl,naive_mac(key,length_extend(tag,kl,msg,ext)[0])==length_extend(tag,kl,msg,ext)[1]]
for kl in(4,8,10,12,16)]
# generic hmac cross-check against stdlib
mk=os.urandom(20); mm=os.urandom(100)
R['q2_hmac_ok']=hmac_generic(lambda
d:hashlib.sha256(d).digest(),64,mk,mm)==hmac.new(mk,mm,'sha256').digest()
H8=lambda d:md(d); hm=lambda k,m:hmac_generic(H8,BLK,k,m)
R['q2_hm']=[hm(key,msg).hex()]
```

```
# tamper scenarios
def scen():
    rows=[]
    good_plain=md(msg); good_naive=naive_mac(key,msg); good_hm=hm(key,msg)
    # S1 modify msg only
    m1=b'amount=900&to=mallory'
    rows.append(['Modify message
only',md(m1)!=good_plain,naive_mac(key,m1)!=good_naive,hm(key,m1)!=good_hm])
    # S2 modify msg and recompute tag (attacker has no key)
    rows.append(['Modify message and recompute digest/tag (no
key)',False,naive_mac(b'guess',m1)!=naive_mac(key,m1),hm(b'guess',m1)!=hm(key,m1)])
    # S3 append data
    ap=msg+b'&to=mallory'
    f,ft=length_extend(good_naive,len(key),msg,b'&to=mallory')
    rows.append(['Append data (length extension, no key)',False if False else md(ap)!=good_plain,
ft!=naive_mac(key,f), hm(key,ap)!=good_hm])
    # S4 flip one bit of tag
    bt=bytearray(good_hm); bt[0]^=1
    rows.append(['Flip one bit of the tag',True,True,hm(key,msg)!=bytes(bt)])
    return rows
R['q2_scen']=scen()
# verification matrix: does receiver ACCEPT forgery?
f,ft=length_extend(naive_mac(key,msg),len(key),msg,b'&to=mallory')
R['q2_accept']=[['Plain hash (attacker replaces file and digest)',True],['Naive H(key||m) against length
extension',naive_mac(key,f)==ft],['HMAC against the same extended message',hm(key,f)==ft]]
# ===== Q3
pub,priv=rsa_keygen(512); m=b'Transfer 100 to Bob'; s=rsa_sign(priv,m)
pub2,_=rsa_keygen(512)
R['q3_rsa']=[rsa_verify(pub,m,s),rsa_verify(pub,m+b'0',s),rsa_verify(pub,m,s^1),rsa_verify(pub2,m,s),pub[
0].bit_length()]
# textbook RSA forgery
def tsig(m_): return pow(m_,priv[1],priv[0])
m1,m2=12345,67890; s1,s2=tsig(m1),tsig(m2); fs=s1*s2%pub[0]
R['q3_forge']=[m1,m2,pow(fs,pub[1],pub[0])==(m1*m2)%pub[0],(m1*m2)]
# blinding forgery of chosen message
target=424242; r=SR.randrange(2,pub[0]); mp=target*pow(r,pub[1],pub[0])%pub[0]; sp=tsig(mp);
fs2=sp*inv(r,pub[0])%pub[0]
R['q3_blind']=[target,pow(fs2,pub[1],pub[0])==target]
# DSA
P=dsa_params(); p,q,g=P; x=SR.randrange(2,q-1); y=pow(g,x,p); m=b'Dose 10mg'
sg=dsa_sign(P,x,m); _,y2=0,pow(g,SR.randrange(2,q-1),p)
R['q3_dsa']=[p.bit_length(),q.bit_length(),dsa_verify(P,y,m,sg),dsa_verify(P,y,m+b'!',sg),dsa_verify(P,y,
m,(sg[0],(sg[1]+1)%q)),dsa_verify(P,y2,m,sg)]
k=SR.randrange(2,q-1); m1,m2=b'Dose 10mg',b'Dose 90mg'
s_1,s_2=dsa_sign(P,x,m1,k),dsa_sign(P,x,m2,k)
h1,h2=H(m1)%q,H(m2)%q; kr=(h1-h2)*inv(s_1[1]-s_2[1],q)%q; xr=(s_1[1]*kr-h1)*inv(s_1[0],q)%q
R['q3_nonce']=[s_1[0]==s_2[0],kr==k,xr==x,pow(g,xr,p)==y]
# ===== Q6 ElGamal / Schnorr / collision forgery
pe,ge=467,2; xe=127; ye=pow(ge,xe,pe); he=lambda m:H(m)%(pe-1); ke=213; msg=b'Pay Dr A 100'
sge=eg_sign(pe,ge,xe,he(msg),ke)
R['q6_eg']=[ye,sge[0],sge[1],eg_verify(pe,ge,ye,he(msg),sge),eg_verify(pe,ge,ye,he(b'Pay Dr A
900'),sge),eg_verify(pe,ge,ye,he(msg),(sge[0],(sge[1]+2)%(pe-
1))),eg_verify(pe,ge,ye,he(msg),((sge[0]+1)%pe,sge[1]))]
# ElGamal nonce reuse (brute force k for toy p)
m_a,m_b=b'Pay Dr A 100',b'Pay Dr A 500'; ka=101
sa=eg_sign(pe,ge,xe,he(m_a),ka); sb=eg_sign(pe,ge,xe,he(m_b),ka)
kfound=next(kk for kk in range(1,pe-1) if math.gcd(kk,pe-1)==1 and pow(ge,kk,pe)==sa[0])
xfound=[xx for xx in range(pe-1) if eg_sign(pe,ge,xx,he(m_a),kfound)==sa and pow(ge,xx,pe)==ye]
R['q6_eg_reuse']=[sa[0]==sb[0],kfound==ka,xfound]
Ps=dsa_params(); ps,qs,gs=Ps; xs=SR.randrange(2,qs-1); ys=pow(gs,xs,ps); m=b'Open valve 3'
sc=schnorr_sign(Ps,xs,m)
R['q6_sch']=[schnorr_verify(Ps,ys,m,sc),schnorr_verify(Ps,ys,m+b'!',sc),schnorr_verify(Ps,ys,m,(sc[0],(sc
[1]+1)%qs))]
kk=SR.randrange(2,qs-1); sc1,sc2=schnorr_sign(Ps,xs,b'Open valve 3',kk),schnorr_sign(Ps,xs,b'Open valve
4',kk)
xr=(sc1[1]-sc2[1])*inv(sc1[0]-sc2[0],qs)%qs
R['q6_sch_reuse']=[xr==xs]
# collision -> forged signature on 32-bit digest
pubk,privk=rsa_keygen(512)
def sig32(m): return pow(trunc(m,32),privk[1],privk[0])
def ver32(m,s): return pow(s,pubk[1],pubk[0])==trunc(m,32)
tb=time.perf_counter(); good={}; N=1<<17
for i in range(N): good[trunc(b'Pay Dr. Rao the sum of Rs 1,000 [ref %d]'%i,32)]=i
hit=None; j=0
while hit is None:
    h=trunc(b'Pay Dr. Mallory the sum of Rs 1,00,000 [ref %d]'%j,32); j+=1
    if h in good: hit=(good[h],j-1)
    if j>(1<<22): break
mb=b'Pay Dr. Rao the sum of Rs 1,000 [ref %d]'%hit[0]; me=b'Pay Dr. Mallory the sum of Rs 1,00,000 [ref
%d]'%hit[1]
sbn=sig32(mb)
R['q6_coll']=[mb.decode(),me.decode(),'%08x'%trunc(mb,32),'%08x'%trunc(me,32),ver32(me,sbn),N,hit[1]+1,ro
und(time.perf_counter()-tb,2),mb!=me]
R['q6_keys']=[[n,n*(n-1)//2,n,n+1] for n in(10,100,1000,10000)]
json.dump(R,open('results3.json','w'),indent=1,default=str)
print('done')
```